# Assignment 3: Image Generation

Theory answers - calculations and explanations

## Question 1: Transposed-convolution output size

Input $I = 8$, kernel $K = 4$, stride $S = 2$, padding $P = 1$, output padding $OP = 1$.
Assume dilation = 1, as no other dilation is specified.

$$
\begin{aligned}
O &= (I - 1)S - 2P + K + OP \\
&= (8 - 1)*2 - 2*1 + 4 + 1 \\
&= 14 - 2 + 4 + 1 \\
&= 17.
\end{aligned}
$$

The output spatial size is 17 x 17.

## Question 2: Increasing the stride

Using the input, kernel, padding and output padding from Question 1,
increase the stride from 2 to 3:

$$
\begin{aligned}
O_{new} &= (8 - 1)*3 - 2*1 + 4 + 1 \\
&= 21 - 2 + 4 + 1 \\
&= 24.
\end{aligned}
$$

The output changes from 17 x 17 to 24 x 24.
In general, increasing stride by 1 increases each output dimension by $I - 1$.
Here the increase is $8 - 1 = 7$ in both height and width.

## Question 3: General output-size formula

For dilation = 1, the transposed-convolution output-size formula is:

$$O = (I - 1)S - 2P + K + OP.$$

Apply the formula independently to height and width.
More generally, if dilation $D$ is included:

$$O = (I - 1)S - 2P + D(K - 1) + OP + 1.$$

With $D = 1$, $D(K - 1) + 1 = K$, giving the first formula.

# Assignment 3: Image Generation

Theory answers - calculations and explanations

## Question 4: Upsampling without padding

One possible configuration: kernel size 2 x 2, stride 2, padding 0, output padding 0, dilation 1.

$$
\begin{aligned}
O &= (16 - 1)*2 - 2*0 + 2 + 0 \\
&= 30 + 2 \\
&= 32.
\end{aligned}
$$

Thus a 16 x 16 feature map becomes 32 x 32.

## Question 5: Batch normalization

Batch: [6, 8, 10, 6]. Use the batch (population) variance, dividing by 4. Gamma and beta are omitted. For this hand calculation, take epsilon = 0; an implementation usually adds a small epsilon for numerical stability.

Mean:

$$\mu = (6 + 8 + 10 + 6)/4 = 30/4 = 7.5.$$

Centered values:

$$[-1.5, 0.5, 2.5, -1.5].$$

Variance:

$$
\begin{aligned}
v &= ((-1.5)^2 + 0.5^2 + 2.5^2 + (-1.5)^2)/4 \\
&= (2.25 + 0.25 + 6.25 + 2.25)/4 \\
&= 11/4 \\
&= 2.75.
\end{aligned}
$$

Standard deviation:

$$\sqrt{v} = \sqrt{2.75} = \sqrt{11}/2 = \text{approximately } 1.658312.$$

Normalized values $\hat{x} = (x - \mu)/\sqrt{v}$:

$$
\begin{aligned}
&[-3/\sqrt{11}, 1/\sqrt{11}, 5/\sqrt{11}, -3/\sqrt{11}] \\
&= \text{approximately } [-0.904534, 0.301511, 1.507557, -0.904534].
\end{aligned}
$$

Check: their mean is 0 and their population variance is

$$(9/11 + 1/11 + 25/11 + 9/11)/4 = 1.$$

# Assignment 3: Image Generation

Theory answers - calculations and explanations

## Question 6: ReLU and LeakyReLU

```
ReLU(x) = max(0, x), equivalently:
  x, if x >= 0;
  0, if x < 0.

LeakyReLU(x) = max(alpha*x, x), for 0 < alpha < 1, equivalently:
  x,       if x >= 0;
  alpha*x, if x < 0.

The key difference is the negative-input branch. ReLU has slope 0 there,
whereas LeakyReLU has slope alpha. At x = 0 they are not differentiable;
frameworks choose a convention for the derivative at that point.
The assignment's discriminator uses alpha = 0.2.
```

## Question 7: Why use LeakyReLU?

```
For negative inputs, ReLU returns 0 and its derivative is 0. If a unit
keeps receiving negative inputs, gradients through it can remain zero,
causing the dying-ReLU problem.

LeakyReLU keeps a small nonzero derivative alpha for negative inputs.
This lets gradients pass through the unit and allows its parameters to
continue changing. It can reduce dead units and help gradient flow in
deep networks, including a GAN discriminator. It does not guarantee
stable training or eliminate every cause of vanishing gradients.
```

## Reference documentation

https://docs.pytorch.org/docs/2.15/generated/torch.nn.ConvTranspose2d.html
https://docs.pytorch.org/docs/2.15/generated/torch.nn.BatchNorm2d.html